



Universidad Central de Venezuela.
Facultad de Ciencias.
Escuela de Computación.
Sistemas Operativos (6004).

Proyecto #2:
Orquestador del Enjambre de Centinelas (The Matrix)

Preparador:
Guillermo Hernandez

Estudiantes:
Almeida Valentina.
CI.32008489
Amselmi Alexandra.
CI.32348685
Matheus Corina.
CI.32104618
Sección 2

Caracas, 25/10/2026.

El Esquema de Planificación (Colas Multinivel Apropiativas):

Estructura queue: Para cumplir en su totalidad con la petición que se realizó de aplicar colas multinivel(hablese de tres niveles, 0,1,2), se investigó sobre las posibles implementaciones que podría tener dicha estructura para los centinelas, se llegó a la conclusión de que la realización de las mismas debía ser a través de nodos, como si fuera una lista enlazada, todo esto porque no se estaba seguro de que tanto podría crecer.
Esto a su vez justifica la creación de el struct Node Centinela, que por lógica, al pensarse como un proceso y siguiendo lo dicho por la bibliografía: "Se enlaza en las colas de listos el PCB asociado a cada proceso" , se sustenta la existencia de dos atributos principales en las mismas:

- PCB.
- Un puntero Node a el siguiente centinela.

La misma, contiene un puntero al inicio y para representar el final del mismo, una variable para contener el tamaño de la estructura y semáforos (estos para prevenir que los hilos implementados más adelante no causen problemas de sincronización ya vistos en la materia) asegurando la mayor simplificación de las funciones posteriormente realizadas.
Nota: Los nombres de las funciones y estas estructuras (como la ya mencionada struct) se encuentra en la carpeta include específicamente el archivo "queue.h"

Funciones implementadas para las colas multinivel:

- void inicializarQueue(Queue* q): Es necesaria para, como su nombre lo indica, inicializar la cola completa y los semáforos a utilizar (una de ellas vista en la materia y otras las habíamos visto en AyED).

- bool isEmpty(Queue * q): Necesaria para verificar si la cola se encuentra vacía, verificando que el inicio de la cola no sea nula, puesto que si lo es retorna verdadero ya que la cola estaba vacía.

- void enqueue(Queue* q, PCB* centinela): Necesaria para lograr ingresar otro centinela a su respectiva cola, tal y como una, la inserción se realiza al final de la misma. Se realiza la creación de un nuevo nodo en la cola, una vez que se verifica que este nodo efectivamente existe usando los parámetros se inicializan los campos de este nodo, de esta manera si el inicio de la cola no existe, entonces éste será dicho inicio, de otro modo simplemente se la asigna el nuevo puntero del final para registrar su entrada, finalmente incrementado así el tamaño.

- PCB* dequeue(Queue* q): Necesaria para retirar centinelas de las colas(bien sea por eventos bloqueantes o terminación de este), una vez que se confirma que la cola no está vacía se asigna un nodo temporal para así liberar la memoria de este nodo y poder cambiar con el temporal el nodo en el inicio para su siguiente centinela.

- int size(Queue* q): Retorna el tamaño de la cola.

- void colocarfrente(Queue* q, PCB* centinela): Necesaria para la implementación del desalojo de centinelas, la misma inserta centinelas al frente de la cola creando un nuevo nodo y cambiando el puntero que indica el inicio de la estructura.

- void destruirQueue(Queue * q): Libera la memoria de cada nodo y destruye los semáforos usados, dejando limpia la estructura para cada nuevo uso. Necesaria para como su nombre lo indica liberar la memoria asignada a nodos que ya no se usarán, incluyendo así los semáforos usados para proteger las regiones críticas.

Nota: La implementación de esto justifica la existencia de el archivo "queue.c" en el apartado src. Todo lo realizado, técnicas, implementaciones, y desarrollo de estos archivos y funciones fue basado en lo visto en la materia para la implementación de semáforos y las demás en conceptos proporcionados en AyED.

Lógica de Expropiación por Prioridad y Desempate:

Para poder aplicar la lógica de dicho despachador y la planificación para cada cola, se crearon cada una de las solicitadas colaPrioridad0 (Diseñada para maniobras tácticas de respuesta rápida. Utiliza el algoritmo Round Robin (RR) con un quantum de Q0 = 2 tics.Tiene prioridad absoluta sobre las demás colas), colaPrioridad1(Diseñada para tareas operativas balanceadas. Utiliza el algoritmo Round Robin (RR) con un quantum de Q1 = 4 tics. Solo se planifica si la Cola 0 está vacía), colaPrioridad2(Diseñada para cómputo pesado de soporte estratégico. Utiliza el algoritmo First-Come, First-Served (FCFS) puro. Solo se planifica si las colas 0 y 1 están completamente vacías).

Para que estas puedan ser manipuladas a lo largo del algoritmo, se debía reservar la memoria que cada una iba a utilizar, por ello se creó void inicializarScheduler() que inicializa todas las colas creadas(razón por la cual se usa void inicializarQueue(Queue* q)). Por otro lado, para poder identificar si la CPU está libre o no de centinelas y si lo está de que cola proviene, para más adelante usar esta información, se definen dos variables básicas:

- static PCB* centinelaCPU: Será NULL si el CPU está libre y tendrá el centinela que el CPu tenga si el mismo está ocupado, específicamente su PCB para poder cambiar el estado del mismo según sea la traza en su ciclo de vida.
- static int colaOrigen : Será -1 si no hay nadie en el CPU, no hay cola origen a registrar. indica de qué cola proviene el centinela que ocupa la CPU.

Nota: Ambas estáticas para que dichas variables conserven su valor en cada llamada que el otro hilo por su parte haga.

Los centinelas leídos llegan una vez que el parser se encarga de procesar la entrada, de manera que su PCB ya tengan los datos correspondientes, una vez que llegan se verifica el siguiente condicional:

if (centinelaCPU != NULL && colaOrigen > 0 && ((colaOrigen == 1 && !isEmpty(&colaPrioridad0)) || (colaOrigen == 2 && (!isEmpty(&colaPrioridad0) || !isEmpty(&colaPrioridad1))))): El inicio en el ya mencionado condicional verifica, si se encuentra un centinela ocupando la CPU y llega un centinela a la cola de más alta prioridad(cola 0 o cual sea según llegue), se verifica según el número de cola de origen y se vuelve a encolar a él principio(void colocarfrente(Queue* q, PCB* centinela)) para que este una vez no este el de alta prioridad, según sea el contexto vuelva desde donde inició(Fase de Despacho (Dispatch) y Fase de Expropiación (Preemption) , por medio de condicionales), liberando así el CPU, actualizando las variables mencionadas con su valors correspondiente.

Una vez este último se encuentra libre, utilizando el bloque de código contenido en el condicional if (centinelaCPU == NULL), formado por condiciones anidadas que meramente verifican si las colas no están vacías(en orden de prioridad desde la cero hasta la 2), de manera que se la asigne, actualizando las variables básicas con la información correspondiente, de el centinela de la cola de más alta prioridad, no vacía , que será asignado oficialmente al CPU.

Luego para poder dar entrada a la Fase de Expiración de Quantum y Rotación, se usaron los bloques condicionales if (centinelaCPU != NULL) , formado también por condicionales anidados e if-else if , estos describen, si el CPU no está vacío, tiene un centinela, se determina si su cola origen es la cola 0 y ya expiró su quantum(q=2), se resetea el valor de dicha rodaja de tiempo, se actualiza su estado para poder volver a la cola de listos, se encola (void enqueue(Queue* q, PCB* centinela)) y liebre el CPU, los mi sucede para la verificación de si viene de la cola 1 y su quántum (q=4) ya expiró repite dicho procedimiento, para la cola 2 no se realizó esta verificación ya que una vez que esta tome el CPU [puede terminar su tiempo y según el orden en el que esté en al cola será asignado, comportamiento descrito en el anterior condicional (FCFS)en caso de que estos todavía cuenten con quantum se les esta de su tiempo de ejecución y se le suma le quetum que ya están consumiendo, por otro lado, si se da el caso de que su tiempo restante es cero, ya terminó su ejecución y puede salir de el sistema, se actualiza su estado, se cataliza su tiempo de retorno según lo que va del click actual, se liebre la CPU y se incrementa el siguiente tic(se le suma 1 ya que al ser un contador, se debe contar ese último tic, ay qu el centinela finaliza en ese tic oficialmente), se hace un return NULL para que el otro hilo tenga el control de la siguiente llamada para el siguiente tic.

Nota: En el anterior texto se refleja el Mecanismo de Desempate y Alternancia (Misma Prioridad). Todo está sustentado en condiciones ya vistos y básicas la única complejidad fue el cómo manejar la colas y esto es algo que según la teoría que ya habíamos manejado en la materia por lo que no se requiere de más ayuda que los libros y textos usados en la materia y la comprensión de los conceptos. Incluyendo prueba y error con respecto a la oportunidad de cuándo sacar o no un centinela para que los tiempos fueran los pertinentes.Está contenido en “scheduler.c” en la carpeta src. La existencia de calcular métricas está justificada por todo lo visto en la materia, las fórmulas son las que se encontraron en los libros y apuntes sobre cada una.

Estructura del Archivo de Entrada:

Se implementó un archivo .c con la función correspondiente a la extracción de los datos proporcionados en el archivo de entrada, para ello se usaron las funciones estándar de C fopen y fclose, atendiendo al principio de que un archivo se abre, se lee y se cierra, una vez se realizaron las correspondientes verificaciones de existencia y apertura del archivo, se extrajo la siguiente información dependiendo de los tres casos que se poseían:

1- Si la línea correspondía a un salto de línea, un espacio o tabulación, entonces dicha línea es ignorada completamente.

2- Si la línea inicia con un "#" entonces se interpreta como un simple comentario y la línea será completamente ignorada.

3- En caso de no caer en los casos anteriores, se procede a declarar valores temporales para extraer usando la función "sscan()" cada uno de los valores según como se especificaba en el formato de las instrucciones, siendo los valores recopilados: el tiempo de llegada, el pid del proceso, su nivel de prioridad y su tiempo de ráfaga. Dichos datos se verifican en cantidad (si no se reciben los 4 parámetros, no se carga) y posteriormente se insertan en el PCB de cada proceso; además si la cantidad de procesos se excede del límite inicial de la estructura, el cual se definió como 128 centinelas (lo que vimos como un número adecuado para el proyecto) entonces se expande multiplicando por 2 la capacidad de la lista de procesos y se continúa con la inserción.

3.1- La estructura del PCB se encuentra en un struct en el archivo con el mismo nombre, teniendo todos los datos correspondientes a la imagen del proceso, en este se encuentran el identificador del proceso, su nivel de prioridad, el estado, tiempos de llegada, rafaga, espera. retorno y restante, quantum consumido. A su vez, el estado se encuentra en un enumerado para restringir los valores que puede tomar y facilitar su uso para la inserción y los cambios de estado, se mantienen dentro los valores indicados en el enunciado: NEW, READY, RUNNING, INTERRUPTED y FINISHED.

A su vez, el nombre del archivo y la ubicación de su ruta, se extraen del argumento de compilación el cual se obtiene con "argv" al inicio del Main y se envía a la función correspondiente.

Mecanismo de Ejecución y Barrera de Fase Síncrona:

Hilo CPU: Llamada void* hilo_cpu(void* arg), permite separar la lógica de cómputo de la CPU y la gestión del reloj maestro, emulando el modelo concurrente de un sistema operativo real. El parámetro arg cumple con la firma estándar de hilos POSIX en lenguaje C para asegurar la compatibilidad con el motor de simulación multihilo. Internamente, esta función administra el ciclo de vida de los procesos y ejecuta la planificación por colas multinivel, manejando de forma automática la expropiación por prioridad, la rotación por quantum en las colas superiores y la ejecución continúa en la cola inferior. Para cumplir con lo solicitado de el manejo de los tic se crea una variable llamada int ticActual= 0 para los tics globales. esta se encuentra en "scheduler.c"

Hilos y sincronización:

Se implementó la lógica de los hilos en un archivo llamado sync.c, utilizando una función para cada hilo: void *hilo_maestro(void *arg) que representa el reloj y la llegada de interrupciones y void *hilo_trabajador(void *arg) que representa la CPU. Una función llamada int sync_run(int verbose) crea los hilos, se asegura que fueron creados sin errores, llama a estas funciones, espera que ambas terminen y libera sus recursos. Para la sincronización se usaron dos semáforos, ambos inicializados en 0:

- static sem_t sem_admision: El maestro lo señala cuando terminó de admitir las llegadas del tic actual, el trabajador no puede ejecutar la CPU hasta recibirlo.
- static sem_t sem_tic: El trabajador lo señala cuando terminó de procesar el tic. El maestro no avanza al siguiente tic hasta recibirlo.
- Como ambos empiezan en 0, el primer tic siempre comienza con la admisión y ningún hilo puede adelantarse al otro.

*hilo_maestro(void *arg) llama a una función llamada static void admitir_llegadas(int tg) la cual encola los centinelas cuyo tiempo de llegada ya se cumplio. *hilo_trabajador(void *arg) ejecuta la funcion hilo_cpu explicada anteriormente. Además cuenta los procesos en estado FINISHED, sí son todos activa una bandera llamada fin, si el hilo maestro lee fin activado termina, sino itera al siguiente tic.

sync_run, crea un arreglo llamado orden con los índices de los procesos y los ordena con qsort usando cmp_llegada, de esta manera si el archivo de entrada no está ordenado, igualmente los centinelas que llegan en el mismo tic se encolan en el orden que aparecen.

Impresiones:

- **Salida en Modo Estándar**:

  En el modo estándar, la impresión solicitada para la salida, según el formato que se especifica en las instrucciones consta de una impresión general de todos los hitos críticos y estadísticos presentados, es decir, un resumen de cómo operaron los centinelas, esto se encuentra en el archivo de "**impresion.c**" en la función de "**impresionCompleta**" en la cual primero se hace la verificación de procesos mayores a 0, debido a que se emplean múltiples divisiones donde este parámetro es denominador, por lo cual, esta corroboración es necesario para que no acaben en errores y valores indeterminados; así mismo se cuentan los tiempos de espera de cada uno de los procesos y se realiza el promedio, a su vez, se calcula el throughput, correspondiente al valor de la cantidad de procesos completados por unidad de tiempo y finalmente la cantidad de tics de tiempo que tomó todas la ejecuciones. Por otro lado, "**impresionIndividual**" se va a encargar en este modo y en verbose de hacer las impresiones de los eventos realmente importantes y el estado en que se encuentran, que ocurrió, entre otros.

- **Salida en Modo Depuración (Verbose)**:

  Este modo va a imprimir las trazas internas una a una con toda la información detallada de los estados de las colas, la decisión que toma el planificador en ese momento, los consumos de quantum y las expropiaciones que se realizan en algunos tics de reloj; esto se encuentra en el archivo de "**impresion.c**" en la función de

"**impresionVerboseTick**", ya que no se poseía un ejemplo en las instrucciones, se optó por imprimir la información actual de cada una de las colas, siendo: quienes se encuentran en cada cola, el tick, la cantidad de procesos activos, su tipo de proceso, si se encuentra vacía, entre otros; de modo que la mayor parte de los datos recopilados en el scheduler y en sync se muestren a forma de estado actual del sistema. A su vez, se incluyeron dos funciones para capturar el modo verbose y enviarlo a estas funciones para comprobar si se debe o no se debe realizar la impresión dependiendo de si está la bandera exacta "[-v]" o "[-verbose]".

Todas estas funciones de impresión se exportan y se utilizan en el Main, en el scheduler y en el sync, dependiendo del momento en donde se requiere la impresión. Además, en el Main se realizan todas las comprobaciones necesarias para saber en qué modo se encuentra actualmente, es decir, verbose o estándar, dependiendo del argumento extraído con "argv". Para asegurar que el mensaje de COMPLETED se imprimiera después del mensaje de un ARRIVAL con la misma marca de tiempo como se muestra en el enunciado, se declaró en scheduler.c una variable estática llamada terminadoPendiente y se implementó una funcion imprimirFinalizacionPendiente, la cual imprime el mensaje al inicio de la siguiente llamada a hilo_cpu consiguiendo así que él COMPLETED se imprima siempre de último, de igual manera el hilo_trabajador llama a esta función cuando la variable fin se activa para imprimir el último COMPLETED.

Manejo de Errores:

Adicionalmente, los errores en todas las secciones de código se corroboran con comprobaciones en estructuras condicionales, para luego ser notificadas con "perror()", se libera la memoria dinámica que fue reservada para los procesos, de forma que todo quede bien finalizado y se retorna un 1 en el main, indicando una finalización con errores, lo cual hace que no se pueda continuar con la corrida del algoritmo de planificación según las directrices del proyecto.